CLAUDE HANDOFF BRIEF

AI Aurum - Intelligent Assistant Platform

Read this entire document before touching any file.

Owner: Yuvan | Tasks done: 104 | July 02, 2026

0. How to Use This Document

You are Claude. The previous Claude session built AI Aurum and ran out of context. This document gives you full project context so you can continue immediately. Read every section before writing any code.

CRITICAL - File Writing Rule:

Never use the Edit or Write tools on files in the project folder. The Windows path with spaces and parentheses silently truncates files. Always write files like this:

```
# Save to a temp script first, then run it:
cat > /tmp/write_file.py << PYEOF
with open('/sessions/.../mnt/AssistNeo-main (1)/myfile.py', 'w') as f:
f.write(content)
PYEOF
python3 /tmp/write_file.py
```

1. What Is AI Aurum

AI Aurum is a full-stack intelligent AI assistant platform. Flask backend, streaming SSE chat UI, 11 specialist AI agents, autonomous task planning, 30+ tools, dynamic plugin system, voice I/O (Whisper STT + ElevenLabs TTS), 5-tier memory, Playwright browser agent, and a comprehensive analytics + observability layer.

Runs at http://localhost:5000 via: python app.py

AI backend: GitHub Models (Azure) using GITHUB_TOKEN env var. Primary model: gpt-4o.

Local fallback: Ollama (llama3.2, mistral). Model routing auto-selects per request type.

2. Folder Structure

Path	What it contains
app.py	20 Flask Blueprints + Limiter + scheduler + session guard + auto-learn job
routes/ (20 files)	auth, chat, stream, research, upload, tools, files, settings, admin, debate, agents, planning, workflow, work
services/ (18 files)	auth, ai, speech, chat, event_bus, capability_registry, agent_health, memory_layers, sandbox, tracer, voic
agents/ (12 files)	11 agents + CEO orchestrator + run_team() in __init__.py
planning/	planner.py (create_plan + execute_plan) + executor.py (SSE stream)
tools/ (30+ files)	Each has NAME, DESCRIPTION, INPUTS, run() - auto-discovered at startup
plugins/	Drop any .py with NAME/DESCRIPTION/run() - auto-loaded, zero config
workflows/	engine.py - trigger/tool/agent/condition/wait steps, SQLite-persisted
assistant/ (9 files)	Refactored monolith, backward compatible

templates/index.html	Single-page app - SSE streaming chat, SVG icons, PWA manifest
db.py	All SQLite ops - users, chats, memory, skills, analytics, projects
vector_memory.py	ChromaDB semantic memory - store + retrieve relevant past chats
agent.py	ReAct tool-calling loop - run_stream() used by /stream endpoint
self_eval.py	Rates AI responses 0-1, hint stashed in session for next turn
push.py	Git push helper - auto-removes .git/index.lock if stuck

3. Blueprint Pattern - Follow Exactly

Every route file uses this exact pattern with _deps dependency injection:

```
from flask import Blueprint, request, jsonify, session
from services.auth_service import login_required
my_bp = Blueprint("myname", __name__)
_deps: dict = {}
def _init(deps: dict): _deps.update(deps)
@my_bp.route("/myroute", methods=["POST"])
@login_required
def my_route():
db = _deps["db"] # injected at startup
data = request.get_json(force=True) or {}
return jsonify({"result": "..."})
```

Register in app.py at the bottom like all the others:

```
from routes.my_routes import my_bp, _init as _my_init
_my_init({"db": _db})
app.register_blueprint(my_bp)
```

4. Agent System

Method on BaseAgent	What it does
think(task, context)	Calls model, records health metrics, fires event bus. Returns text.
scored_think(task, context)	Returns dict: {answer, confidence, reasoning_quality, agent, model}
_call_tool(name, **kwargs)	Routes to tools/__init__.py call(). Returns dict.

CEO agent (agents/ceo_agent.py) builds agent list from CapabilityRegistry - NOT hardcoded. GPT-chosen agent names that do not match registry get remapped automatically to best match. CEO returns: {"plan": "...", "steps": [{"agent": "researcher", "task": "..."}], "single_agent": false}

Agent	Role	Model
ceo	Routes goals, synthesises results from all agents	gpt-4o
planner	Step-by-step plans, time estimates	gpt-4o-mini
researcher	Web search, fact-finding, data gathering	gpt-4o
programmer	Writes, reviews, debugs code	gpt-4o
debugger	Diagnoses bugs, traces errors	gpt-4o-mini
reviewer	Code review, quality checks	gpt-4o-mini
memory_manager	Stores/retrieves long-term knowledge	gpt-4o-mini
vision	Analyses images, engineering drawings	gpt-4o
automation	Scripts, scheduled tasks, workflows	gpt-4o-mini
browser	Web scraping, form filling via Playwright	gpt-4o
security	Vulnerability scanning, auth review	gpt-4o-mini

5. Key Services (all are singletons)

File	Singleton var	Key API
event_bus.py	bus	bus.emit("agent.completed", data, async_=True)

capability_registry.py	registry	registry.rank(task, top_n=3) -> [(name, score)]
agent_health.py	health	health.record(name, latency_ms, success); health.pick_healthy(candidates)
memory_layers.py	mem	mem.context_string(username, query) -> str for system prompt
sandbox.py	sandbox	sandbox.run_plugin(name, isolate=False, **kwargs) -> dict
tracer.py	tracer	tracer.start_trace(username); tracer.start_span(tid, name); tracer.end_span(sid)
auth_service.py	-	@login_required decorator; hash_password(); check_password()
ai_service.py	-	route_model(msg, settings) -> "code"/"fast"/"vision"/"reason"/"main"

6. The /stream Route Pipeline

Step	What happens
1 Auth	login_required blocks unauthenticated requests
2 Trace start	tracer.start_trace() assigns trace_id
3 Model select	route_model() picks code/fast/vision/reason/main model
4 Memory	db.get_memories() + vmem.retrieve_relevant() + mem.context_string() (5-tier)
5 Project ctx	db.get_project_context() injected into system prompt
6 Skills ctx	db.search_skills() - relevant saved skills prepended
7 Eval hint	session.pop("_eval_hint") from last turn injected
8 Generate	agent.run_stream() - SSE tokens streamed to browser
9 Save	db.save_chat() + vmem.store_conversation()
10 Self-eval	self_eval.evaluate() rates response, stashes hint for next turn
11 Trace end	tracer.end_trace() closes the span
12 Done SSE	{done:true, chat_id, title, model, trace_id, eval_warning?}

7. Memory System - 5 Tiers

Tier	Class	Storage	Purpose
1 Working	WorkingMemory	In-process TTL dict	Fast same-session data
2 Conversation	ConversationMemory	SQLite 30-day TTL	Recent chat history
3 Knowledge Graph	KnowledgeGraphMemory	NetworkX + SQLite	Entity relationships
4 Vector	VectorMemory	ChromaDB embeddings	Semantic search
5 Archive	ArchiveMemory	SQLite FTS5	Cold full-text search

Call mem.context_string(username, query) to get a formatted string ready to inject into the system prompt. Already wired into /stream automatically.

8. Tools System

Each tool in tools/ has: NAME, DESCRIPTION, CATEGORY, ICON, INPUTS list, run(**kwargs) -> dict. Auto-discovered at startup. call(name, **kwargs) routes execution. Metrics recorded per call.

Tool	Capability
electrical_engineering	IEC cable sizing, transformer, fault level, solar PV, BESS, tariff
meeting_assistant	Whisper transcribe + summarise + action items + minutes + email draft
dev_agent	Multi-file project builder: programmer + reviewer + debugger in loop
knowledge_graph	NetworkX + SQLite entity/relation graph. add_entity, add_relation, query
web_search	Gemini grounding + news/research/price/compare modes
vision_tool	GPT-4o vision for images, P&IDs, engineering drawings
browser_tool	Playwright: navigate, click, fill forms, scrape, screenshot
system_monitor	CPU/RAM/GPU/temp + threshold alerts
code_runner	Python execution in subprocess (admin only)
document_agent	PDF/DOCX/PPTX/Excel/OCR unified analysis

9. Active API Routes (20 Blueprints)

Route	Method	Purpose
/stream	POST	SSE streaming chat with tools, memory, tracing, self-eval
/agents/run	POST	CEO multi-agent team SSE stream
/agents/health	GET	Health scores for all 11 agents
/agents/best	POST	Best agent for a task (registry + health scoring)
/plan	POST	Autonomous planning + execution SSE stream
/debate	POST	Multi-model debate SSE stream
/vote	POST	Model voting consensus SSE stream
/workflows	POST	Create and run AI workflows
/traces	GET	List recent request traces (SQLite)
/traces/<id>/graph	GET	Execution DAG for a trace
/voice/transcribe	POST	Whisper speech-to-text

/dashboard/stream	GET	Live system metrics SSE
/analytics	GET	Performance analytics dashboard
/tools/run	POST	Execute any registered tool
/research	POST	Deep web research mode
/workspace/projects	GET	List/manage user projects
/memory	GET	User memory facts
/admin/metrics	GET	Admin metrics (admin role only)

10. Critical Rules - Do Not Break

1. File writing

ALWAYS write files via bash python script, NEVER Edit/Write tools. Windows path truncation bug.

2. Python syntax check

After writing any .py file run: python3 -m py_compile filepath.py and fix before moving on.

3. JS syntax check

After editing index.html extract script blocks and run node --check. One error breaks everything.

4. No duplicate JS functions

Never define function addActions() twice. JS hoisting causes infinite recursion.

5. Style names no spaces

ReportLab ParagraphStyle names cannot contain spaces. Use numeric counters instead.

6. app.py tail check

app.py is prone to truncation. Always check: tail -10 app.py to confirm the run block exists.

7. Git lock

push.py auto-removes .git/index.lock. Manual fix: del .git\\index.lock in CMD.

8. Unicode in f-strings

No em-dashes or special Unicode inside f-strings. Use plain ASCII hyphens only.

11. Current Project Status (After 104 Tasks)

Component	Status
app.py	20 blueprints registered. Flask-Limiter. APScheduler. Auto-learn at 03:00. Session age guard.
Agents	11 agents. CEO uses CapabilityRegistry (not hardcoded). /agents/health and /agents/best live.
Memory	5-tier system. mem.context_string() injected into every /stream system prompt automatically.
Tracing	Every /stream request gets trace_id. Spans in SQLite. /traces and /traces/<id>/graph live.
UI icons	All emojis replaced with inline SVG icons in plus menu, input bar, and panel headers.
Enter key	Fixed - was broken by corrupted JS catch block (two fragments merged into one line).
Infinite loop	Fixed - addActions() was defined twice causing hoisting recursion. Now single definition.
push.py	Auto-removes git index.lock at startup. Safe to run anytime.

12. How to Start Your Session

Paste this message to get oriented immediately:

```
"I am continuing AI Aurum. I have read the handoff brief.
Project is at: C:\\Users\\shakt\\Downloads\\AssistNeo-main (1)\\
104 tasks done. Flask app with 20 blueprints, 11 agents, 5-tier memory,
observability tracing, and full plugin system.
[Tell me what you want to do next]"
```

Useful verification commands for a new session:

Check	Command
App boots	python app.py (look for "Starting AI Aurum on port 5000")
Python syntax sweep	for f in routes/*.py services/*.py agents/*.py: python3 -m py_compile "$f"
JS syntax	Extract script block from index.html, save as .js, node --check it
Registered blueprints	grep register_blueprint app.py

All tools	python3 -c "import tools; print(list(tools._REGISTRY.keys()))"
All agents	python3 -c "import agents; print(list(agents.REGISTRY.keys()))"